

Lab 3

Before now the entire game lived in a single `Program.Main` method — roughly 200 lines of SDL initialization, window management, event polling, and rendering all tangled together. This lab breaks that apart into four focused classes, each owning one concern.

GameWindow

The first extracted piece is window management. `GameWindow` takes the SDL instance in its constructor and immediately creates the OS window:

```
_window = (IntPtr)sdl.CreateWindow(
    "The Adventure", Sdl.WindowposUndefined, Sdl.WindowposUndefined, 800, 800,
    (uint)WindowFlags.Resizable | (uint)WindowFlags.AllowHighdpi
);
if (_window == IntPtr.Zero)
{
    var ex = sdl.GetErrorAsException();
    if (ex != null) throw ex;
    throw new Exception("Failed to create window.");
}
```

It also exposes a `CreateRenderer()` method and a `Destroy()` method, keeping all raw window handle manipulation — including the unsafe pointer casts — contained within this one class.

GameRenderer

`GameRenderer` is responsible for presenting frames to screen. It acquires its renderer through `GameWindow.CreateRenderer()` at construction time, then exposes a single `Render()` method:

```
public GameRenderer(Sdl sdl, GameWindow gameWindow, GameLogic gameLogic)
{
    _sdl = sdl;
    _renderer = gameWindow.CreateRenderer();
}

public void Render()
{
    _sdl.RenderPresent((Renderer*)_renderer);
}
```

The unsafe pointer cast that was previously scattered in `Main` is now fully hidden from callers. `GameLogic` is accepted as a constructor parameter but not used yet — the renderer will eventually pull state from game logic to decide what to draw.

GameLogic

`GameLogic` is the most minimal of the four:

```
public class GameLogic
{
    public void ProcessFrame()
    {
    }
}
```

It's an empty stub, but its presence in the game loop is what matters. By calling `gameLogic.ProcessFrame()` every tick, the architecture is ready to handle state updates that can come in later labs.

InputLogic

`InputLogic` takes over the SDL event polling loop. Its `ProcessInput()` method drains the SDL event queue every frame and returns true when the application should quit:

```
public bool ProcessInput()
{
    ReadOnlySpan<byte> _keyboardState = new(_sdl.GetKeyboardState(null),
(int)KeyCode.Count);
    Span<byte> mouseButtonStates = stackalloc byte[(int)MouseButton.Count];

    Event ev = new Event();
    while (_sdl.PollEvent(ref ev) != 0)
    {
        if (ev.Type == (uint)EventType.Quit) return true;

        switch (ev.Type)
        {
            case (uint)EventType.Mousebuttondown:
                mouseButtonStates[ev.Button.Button] = 1;
                break;
            case (uint)EventType.Mousebuttonup:
                mouseButtonStates[ev.Button.Button] = 0;
                break;
            case (uint)EventType.Windowevent:
                // ... including the one non-trivial case:
                case (byte)WindowEventID.TakeFocus:

                _sdl.SetWindowInputFocus(_sdl.GetWindowFromID(ev.Window.WindowID));
                break;
        }
    }
    return false;
}
```

An interesting note is that `mouseButtonStates` is `stackalloc'd` — allocated on the stack rather than the heap, avoiding GC pressure for something that's created and thrown away every single frame. Most event handlers are empty break cases for now, but the structure is in place for all the input types the game will eventually need: keyboard, mouse, gamepad, and window focus events.

Program.Main

With all four classes in place, the game loop in `Main` collapses to a clear, readable sequence of intent:

```
var gameWindow = new GameWindow(sdl);
var gameLogic = new GameLogic();
var gameRenderer = new GameRenderer(sdl, gameWindow, gameLogic);
var inputLogic = new InputLogic(sdl);

while (!quit)
{
    quit = inputLogic.ProcessInput();
    if (quit) break;
    gameLogic.ProcessFrame();

    var elapsed = timer.Elapsed;
    timer.Restart();

    gameRenderer.Render();
    ++framesRenderedCounter;
}

gameWindow.Destroy();
```

`Main` no longer needs to know anything about SDL handles, pointer casts, or event types. It just coordinates the four classes in the right order each frame.

Next we introduce a game object hierarchy, image loading, and a working render pipeline — for the first time, something actually appears on screen.

The object hierarchy — `GameObject` and `RenderableGameObject`

Two new classes form the foundation of everything that will exist in the game world. `GameObject` is a bare base class:

```
public class GameObject
{
}
```

Empty for now, but its purpose is to give all game entities a common type so they can be stored and iterated together. `RenderableGameObject` extends it with everything needed to draw something:

```
public class RenderableObject : GameObject
{
    public int TextureId { get; }
    public Rectangle<int> TextureSource { get; set; }
    public Rectangle<int> TextureDestination { get; set; }
    public GameRenderer.TextureInfo TextureInformation { get; }
}
```

`TextureId` is a handle into the renderer's texture registry. `TextureSource` is the rectangle within the source image to sample from — crucial for sprite sheets, as we'll see shortly. `TextureDestination` is where on screen to draw it. Both are mutable (set) because they change every frame.

GameRenderer — loading and drawing textures

`GameRenderer` gains two significant additions. First, a `TextureInfo` struct is introduced to carry image metadata:

```
public readonly struct TextureInfo
{
    public int Width { get; init; }
    public int Height { get; init; }
    public int PixelDataSize => Width * Height * 4; // RGBA, 4 bytes per pixel
}
```

Second, a `LoadTexture` method is added, which is where the new `SixLabors.ImageSharp` dependency comes in. It reads an image file, decodes it into raw RGBA pixel data, uploads it to the GPU as an SDL texture, and returns an integer ID that callers can use to reference it later:

```
public int LoadTexture(string fileName, out TextureInfo textureInfo)
{
    using var fStream = new FileStream(fileName, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    var image = Image.Load<Rgba32>(fStream);
    textureInfo = new TextureInfo() { Width = image.Width, Height =
        image.Height };

    var imageRawData = new byte[textureInfo.PixelDataSize];
    image.CopyPixelDataTo(imageRawData.AsSpan());

    fixed (byte* data = imageRawData)
    {
        var imageSurface = _sdl.CreateGBSSurfaceWithFormatFrom(data,
            textureInfo.Width, textureInfo.Height,
            8, textureInfo.Width * 4, (uint)PixelFormatEnum.Rgba32);
        imageTexture = _sdl.CreateTextureFromSurface((Renderer*)_renderer,
            imageSurface);
        _sdl.FreeSurface(imageSurface); // surface is only needed to create the
```

```

texture, free it immediately
    }

    _texturePointers[_index] = (IntPtr)imageTexture;
    _textureInformation[_index] = textureInfo;
    return _index++;
}

```

The texture IDs are stored in two dictionaries — one mapping IDs to raw SDL texture pointers, one to their `TextureInfo` metadata. The fixed block pins the managed byte array in memory so the GC can't move it while SDL is reading from it.

The `Render` method is also fleshed out. Previously it just called `RenderPresent`; now it clears the screen, iterates all renderable objects from `GameLogic`, and draws each one:

```

public void Render()
{
    var renderer = (Renderer*)_renderer;
    _sdl.RenderClear(renderer);

    foreach (var renderable in _gameLogic.GetRenderables())
    {
        if (renderable.TextureId > -1 &&
            _texturePointers.TryGetValue(renderable.TextureId, out var
texturePointer))
        {
            _sdl.RenderCopyEx(renderer, (Texture*)texturePointer,
                renderable.TextureSource, renderable.TextureDestination,
                0, new Silk.NET.SDL.Point(0, 0), RendererFlip.None);
        }
    }

    _sdl.RenderPresent(renderer);
}

```

The `RenderCopyEx` call is doing the actual work: it samples the region defined by `TextureSource` from the texture and draws it at `TextureDestination` on screen. This is the core of sprite sheet rendering.

GameLogic — sprite sheet animation

`GameLogic` goes from a stub to something that actually drives animation. On startup, `InitializeGame` loads the sprite sheet and creates the first game object:

```

public void InitializeGame(GameRenderer gameRenderer)
{
    var textureId = gameRenderer.LoadTexture("image.png", out var textureInfo);
    var sampleRenderableObject = new RenderableGameObject(textureId,
        new Rectangle<int>(0, 0, textureInfo.Width, textureInfo.Height),

```

```
new Rectangle<int>(0, 0, textureInfo.Width, textureInfo.Height),
textureInfo);
_gameObjects.Add(sampleRenderableObject);
}
```

```
public void ProcessFrame()
{
    var renderableObject = (RenderableObject)_gameObjects.First();
    var i = _frameCount % 10; // column
    var j = _frameCount / 10; // row
    var cellWidth = renderableObject.TextureInformation.Width / 10;
    var cellHeight = renderableObject.TextureInformation.Height / 10;

    renderableObject.TextureSource = new Rectangle<int>(i * cellWidth, j *
cellHeight, cellWidth, cellHeight);
    renderableObject.TextureDestination = new Rectangle<int>(0, 0, cellWidth,
cellHeight);

    if (++_frameCount == 100) _frameCount = 0;
}
```

Program.cs

```
gameLogic.InitializeGame(gameRenderer);

while (!quit)
{
    quit = inputLogic.ProcessInput();
    if (quit) break;
    gameLogic.ProcessFrame();
    // ...
    gameRenderer.Render();
    ++framesRenderedCounter;

    System.Threading.Thread.Sleep(TimeSpan.FromSeconds(0.41666666666666666666666666666667));
}
```

```
}
```

New NuGet dependency

`SixLabors.ImageSharp` is added to the project for image decoding, and `image.png` is added as a content file configured to be copied to the output directory:

```
<PackageReference Include="SixLabors.ImageSharp" Version="3.1.12" />

<Content Include="image.png">
  <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
</Content>
```

This ensures the image is available next to the executable at runtime without needing to hardcode an absolute path.

In this lab we broke apart the monolithic `Main` method into logical parts. And maybe most importantly expanded the game loop to load a real image, advance through its frames on every tick, and render the current frame to screen.